Loan Default Prediction

Comparing Classic ML and Neural Networks under Class Imbalance

Minwoo Yoo Nathaniel Badalov

Home Credit Default Risk · Kaggle

Problem & Motivation

The task

Predict the probability that a loan applicant defaults (binary, `TARGET` $\in \{0, 1\}$). **Why class imbalance matters**

- Only $\sim$8.07% of applicants default — the data is severely imbalanced.
- A "predict-no-default" baseline scores 92% accuracy with zero recall.
- A model can hit ROC AUC $= 0.745$ yet classify zero defaulters at the 0.5 threshold.

Research question

Do classic shallow learners and neural networks respond differently to class imbalance handling?

Dataset · Home Credit Default Risk

Size

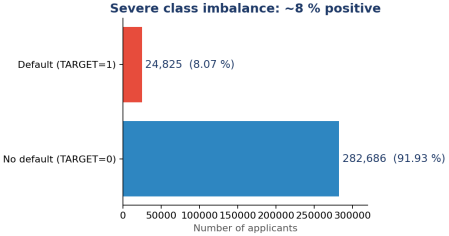
- Train: 307,511 applicants $\times$ 122 features
- Test: 48,744 applicants (Kaggle hidden labels)

Class imbalance

- Default rate $\approx$ 8.07% (24,825 / 307,511)

Strongest signals

- `EXT_SOURCE_1/2/3` (external credit scores), $|\rho| \approx 0.16$–0.18
- `DAYS_EMPLOYED` sentinel 365,243 flags 18% (unemployed)
- Building-info columns 50–70% missing



Methodology · Shared Pipeline

Preprocessing (identical for all five models)

- 80% / 20% stratified train–val split (`random_state = 42`)
- Drop `SK_ID_CURR`; keep IDs only for the submission file
- Mean imputation for numeric NaN; one-hot encode 16 categoricals → 244 features
- `StandardScaler` fitted on train only (essential for LR / MLP)

Tuning & validation

- `GridSearchCV` with `PredefinedSplit` (single train→val fold, no random k-fold)
- Scoring: `roc_auc`; best parameter setting picked per model
- CV CSVs written to `result/home_credit/cv_results/GridSearchCV/`

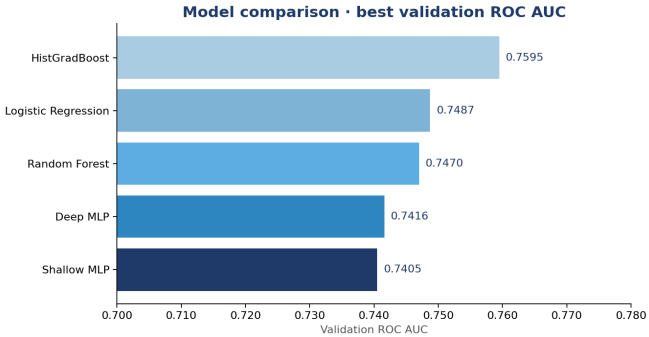
Five Models Compared

Model	Family	Owner	Tuned hyperparameters
Logistic Regression	Linear	Nathan	`C`, `tol`; `class_weight=balanced`
Random Forest	Tree (bagging)	Nathan	`n_estimators`, `min_samples_*`; `class_weight`
HistGradientBoosting	Tree (boosting)	Nathan	`learning_rate`, `max_iter`, `min_samples_leaf`
Shallow MLP — hidden $(64,)$	Neural network	Minwoo	`alpha`, `learning_rate_init`
Deep MLP — hidden $(128, 64, 32)$	Neural network	Minwoo	`alpha`, `learning_rate_init`

Why this lineup?

- Spans the curriculum: linear $\rightarrow$ tree (bagging + boosting) $\rightarrow$ shallow + deep neural networks.
- Same preprocessing, same train/val split, same scoring $\rightarrow$ fair comparison.

Combined Validation ROC AUC



Model	AUC
HistGradBoost	**0.7595**
Logistic Reg.	0.7487
Random Forest	0.7470
Deep MLP	0.7416
Shallow MLP	0.7405

Tree boosting wins by ∼1 AUC point — consistent with the literature on tabular credit data.

Neural Network Track

Architecture

- Shallow MLP — hidden $(64,)$ $\sim$15.7K parameters
- Deep MLP — hidden $(128, 64, 32)$ $\sim$42K parameters
- ReLU activation, Adam optimizer, `early_stopping = True`

Hyperparameter tuning

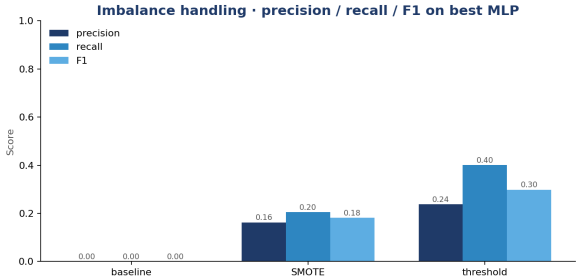
- `alpha` (L2 reg.) $\in \{10^{-4}, 10^{-3}\}$, `lr_init` $\in \{0.001, 0.005\}$
- Best for both: $\alpha = 10^{-3}$, lr $= 0.005$

Findings

- Shallow 0.7405 vs Deep 0.7416 $\Rightarrow$ depth adds $\sim$0.001 (noise).
- NN ranks 4th/5th overall; bottleneck is the feature set, not the model.

Imbalance Handling on the Best MLP

Method	Threshold	ROC AUC	PR AUC	P	R	F1
Baseline	0.500	0.745	0.222	0.00	0.00	0.00
SMOTE	0.500	0.648	0.134	0.16	0.21	0.18
Threshold tuning	0.164	0.745	0.222	0.24	**0.40**	**0.30**



Headline

- Baseline: AUC 0.745, recall **0%** (no prediction crosses 0.5).
- SMOTE **hurt**: AUC dropped 0.10.
- Threshold tuning at 0.164 lifts recall to 40%.

Choosing the threshold matters more than rebalancing the data.

Conclusions

What we learned

- Best: HistGradBoost at val ROC AUC $= 0.7595$; tree boosting wins by $\sim$1 AUC point.
- Neural networks tie LR and RF — depth alone does not help on this tabular data.
- On the MLP, threshold tuning lifts recall from 0% to 40%; SMOTE hurt AUC.

Limitations

- Only main `application_*.csv` tables — auxiliary tables typically add several AUC points.
- `PredefinedSplit` is fast but optimistic; full k-fold CV would tighten confidence intervals.

Future work

- Integrate auxiliary tables; ensemble HGBC + best MLP; Platt / isotonic calibration.

Thank You

Questions?

Recording link: `<<INSERT RECORDING LINK BEFORE SUBMISSION>>`

Code: `github.com/ymw0414/loan-default-imbalance-classification`